

算法设计与分析实验报告

计子毅 2023212872

1 实验任务背景

本实验旨在通过实现四种经典的贪心算法来解决不同的计算问题，包括单源最短路径问题（Dijkstra算法）、最小生成树问题（Prim算法和Kruskal算法）以及Huffman编码问题。这些算法在计算机网络、数据压缩、路径规划等领域具有重要的应用价值。

实验目标包括：

- 实现Dijkstra算法解决单源最短路径问题
- 实现Prim算法和Kruskal算法解决最小生成树问题
- 实现Huffman编码算法进行数据压缩
- 通过测试验证算法性能，分析不同算法的优缺点

2 算法原理与设计

2.1 Dijkstra算法（单源最短路径）

Dijkstra算法用于解决带权有向图或无向图的单源最短路径问题，要求所有权重为非负值。

算法原理：

- 维护一个距离数组 `dis[]`，表示从源点到各顶点的当前最短距离
- 每次选择距离源点最近且未访问的顶点，松弛其邻接顶点
- 使用贪心策略，每次选择局部最优解

算法步骤：

- 初始化距离数组，源点距离为0，其他为无穷大
- 选择未访问顶点中距离最小的顶点u
- 对u的所有邻接顶点v进行松弛操作：`dis[v] = min(dis[v], dis[u] + Graph[u][v])`
- 标记u为已访问，重复步骤2-3直到所有顶点被访问

2.2 Prim算法（最小生成树）

Prim算法用于构建无向图的最小生成树，从单个顶点开始逐步扩展。

算法原理：

- 维护一个集合S，包含已加入生成树的顶点
- 每次选择连接S与V-S的最小权值边加入生成树
- 使用贪心策略保证每次加入的边都是当前最优选择

算法步骤：

- 初始化距离数组，源点距离为0，其他为无穷大
- 选择未访问顶点中距离生成树最近的顶点u

3. 将u加入生成树，更新u的邻接顶点到生成树的距离
4. 重复步骤2-3直到所有顶点加入生成树

2.3 Kruskal算法（最小生成树）

Kruskal算法通过排序和并查集来构建最小生成树。

算法原理：

- 将所有边按权值从小到大排序
- 依次选择不会形成环的边加入生成树
- 使用并查集高效判断是否形成环

算法步骤：

1. 将所有边按权值排序
2. 初始化并查集，每个顶点独立成集合
3. 依次选择最小权值边，如果边的两个顶点不在同一集合则加入生成树
4. 重复步骤3直到生成树包含V-1条边

2.4 Huffman编码算法

Huffman编码用于数据压缩，根据字符出现频率构建最优前缀码。

算法原理：

- 频率越高的字符编码越短，频率越低的字符编码越长
- 通过构建Huffman树实现最优编码
- 使用优先队列（最小堆）选择频率最小的节点合并

算法步骤：

1. 创建n个叶子节点，每个节点包含字符频率
2. 构建优先队列，按频率排序
3. 每次取出频率最小的两个节点，合并为新节点，频率为两者之和
4. 将新节点加入队列，重复步骤3直到只剩一个节点
5. 计算加权路径长度： $WPL = \sum(\text{频率} \times \text{编码长度})$

3 算法实现细节

3.1 Dijkstra算法实现

采用数学归纳法证明Dijkstra算法的正确性。

基础情况：

- 初始时，集合S只包含源点s， $dis[s] = 0$ ，显然是最短距离

归纳假设：

- 假设前k次选择的顶点都已获得最短路径

归纳步骤：

设第k+1次选择顶点u，即u是V-S中距离s最近的顶点。要证明 $dis[u]$ 就是s到u的最短距离。

1. **反证法假设**: 假设存在一条更短的路径 $P: s \rightarrow \dots \rightarrow x \rightarrow y \rightarrow \dots \rightarrow u$, 其中 $x \in S, y \in V-S$, 且 $w(P) < dis[u]$
2. **路径分析**: 由于 s 到 x 的路径完全在 S 中, 根据归纳假设, $dis[x]$ 是 s 到 x 的最短距离
3. **权值非负性**: 因为边权非负, 所以: $w(P) = dis[x] + w(x \rightarrow y) + w(y \rightarrow \dots \rightarrow u) \geq dis[x] + w(x \rightarrow y)$
4. **算法选择矛盾**: 根据算法, 我们选择的是 u 而不是 y , 说明: $dis[u] \leq dis[y] \leq dis[x] + w(x \rightarrow y) \leq w(P)$
5. **矛盾得出**: 这与假设 $w(P) < dis[u]$ 矛盾, 因此不存在这样的更短路径 P

结论: $dis[u]$ 确实是 s 到 u 的最短距离, Dijkstra算法的贪心选择性质得证。

```

1  int dijkstra() {
2      for (int i = 0; i < v - 1; i++) {
3          int node = -1;
4          // 选择未访问顶点中距离最小的顶点
5          for (int j = 1; j <= v; j++) {
6              if (!vis[j] && (node == -1 || dis[j] < dis[node])) {
7                  node = j;
8              }
9          }
10
11         // 松弛操作: 更新邻接顶点距离
12         for (int j = 1; j <= v; j++) {
13             dis[j] = min(dis[j], dis[node] + Graph[node][j]);
14         }
15
16         vis[node] = true;
17     }
18     return dis[v] == inf ? -1 : dis[v];
19 }

```

3.2 Prim算法实现

采用反证法证明Prim算法的贪心选择性质。

1. **假设**: 存在连通带权图 $G=(V,E)$, 设 T 是 G 的一棵最小生成树, 但不包含权值最小的跨割边 (u,v) , 其中 $u \in U, v \in V-U$ 。
2. **构造新生成树**: 由于 T 是生成树, 在 T 中 u 到 v 存在唯一路径 p 。路径 p 上必有一条边 (u',v') 连接 U 和 $V-U$ (因为 $u \in U, v \in V-U$)。设这条边为 $e'=(u',v')$, 其中 $u' \in U, v' \in V-U$ 。
3. **比较权值**: 根据Prim算法的选择, 边 (u,v) 是连接 U 和 $V-U$ 的权值最小边, 因此有: $w(u,v) \leq w(u',v')$
4. **替换构造**: 从 T 中删除边 e' , 加入边 (u,v) , 得到新生成树 T' 。则: $w(T') = w(T) - w(u',v') + w(u,v) \leq w(T)$
5. **矛盾分析**: 如果 $w(u,v) < w(u',v')$, 则 $w(T') < w(T)$, 这与 T 是最小生成树矛盾。如果 $w(u,v) = w(u',v')$, 则 $w(T') = w(T)$, T' 也是最小生成树且包含边 (u,v)
6. **结论**: 因此, 必存在一棵包含边 (u,v) 的最小生成树, Prim算法的贪心选择性质得证。

```

1  int prim() {
2      int res = 0;
3      // 初始化第一个顶点到其他顶点的距离
4      for (int i = 2; i <= v; i++) dis[i] = min(dis[i], G[1][i]);
5
6      for (int i = 2; i <= v; i++) {
7          int t = -1;
8          int temp = inf;
9          // 选择距离生成树最近的顶点

```

```

10     for (int j = 2; j <= V; j++) {
11         if (!vis[j] && dis[j] < temp) {
12             t = j;
13             temp = dis[j];
14         }
15     }
16
17     if (t == -1) return -1; // 图不连通
18
19     res += temp;
20     vis[t] = true;
21     // 更新新加入顶点邻接点的距离
22     for (int j = 2; j <= V; j++) dis[j] = min(dis[j], g[t][j]);
23 }
24 return res;
25 }

```

3.3 Kruskal算法实现

观察Kruskal算法：

无向连通网的最小生成树首先是一棵生成树，即它应该是一个使网中所有顶点相连通而所需边的条数为最小的子网络，且其代价和在所有生成树中为最小。因此，可以从网的连通性角度来观察Kruskal算法。

初始时，最小生成树就是网中所有顶点的集合，它们之间没有任何一条边连接，即它们自成一个连通分量。而Kruskal算法的执行过程其实就是一个选取网中权值为最小的边的过程，即将两个小的连通分量连接为较大的连通分量，直至所有顶点都在一个连通分量中为止。每当选取网中的一条边时总会出现以下两种情况之一：

①该边所依附的两个顶点分属于不同的连通分量。这时，该边可以作为最小生成树的一条边，因为两个不同的连通分量通过这条边的连接而相连通，成为了一个连通分量。

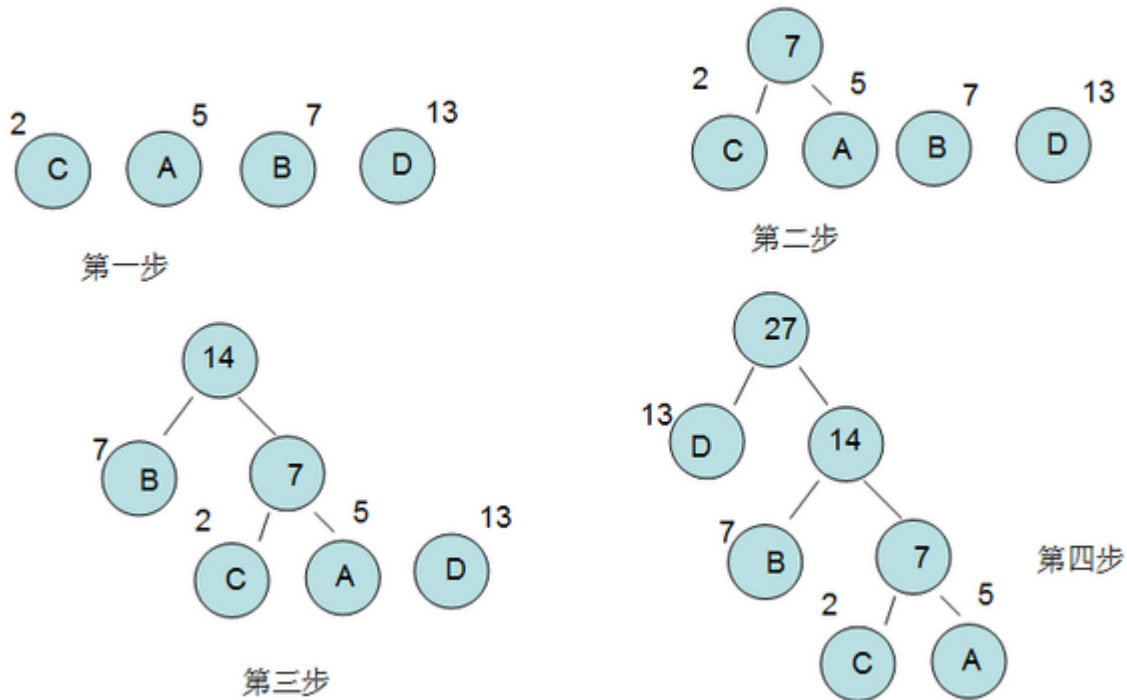
②该边所依附的两个顶点属于同一个连通分量。如果选取这条边作为最小生成树的一条边，则必将构成环。因为连通分量中任意两个顶点间都有路径相通，一旦再加入一条边，该边所依附的两个顶点之间就有了两条路径，即构成了环。故属于这种情况的边即使其权值小也应该舍弃

```

1  int kruskal() {
2      int ans = 0;
3      int cnt = V; // 连通分量计数
4
5      // 按边权排序
6      sort(G + 1, G + 1 + E);
7
8      for (int i = 1; i <= E; i++) {
9          int x = find(G[i].l), y = find(G[i].r);
10         // 如果不在同一连通分量，加入生成树
11         if (x != y) {
12             fa[x] = y;
13             ans += G[i].w;
14             cnt--;
15         }
16         if (cnt == 1) break; // 已形成生成树
17     }
18     return cnt != 1 ? -1 : ans; // 检查是否连通

```

3.4 Huffman编码实现



```

1  double HuffmanTree() {
2      int m = 2 * n - 1; // Huffman树总节点数
3
4      for (int i = n + 1; i <= m; i++) {
5          int n1, n2;
6          select(i - 1, n1, n2); // 选择两个频率最小的节点
7
8          if (n1 == -1 && n2 == -1) break;
9
10         // 合并节点
11         nodes[i].l = n1;
12         nodes[i].r = n2;
13         nodes[i].v = nodes[n1].v + nodes[n2].v;
14
15         nodes[n1].used = true;
16         nodes[n1].p = i;
17         nodes[n2].used = true;
18         nodes[n2].p = i;
19     }
20
21     // 计算加权路径长度
22     double ans = 0;
23     for (int i = 1; i <= n; i++) {
24         int deep = 0;
25         int cur = i;
26
27         while (nodes[cur].p != 0) {

```

```
28         deep++;
29         cur = nodes[cur].p;
30     }
31
32     ans += nodes[i].v * deep;
33 }
34
35 return ans;
36 }
```

4 实验测试与结果分析

4.1 测试环境

- 操作系统: Windows 11
- 编译器: GCC/G++
- 测试数据规模: 符合各题目数据范围要求

4.2 测试用例

测试用例1: Dijkstra算法测试

```
1  输入:
2  V=3, E=3
3  边信息:
4  (1,2,5), (2,3,5), (3,1,2)
5  预期输出: 2
```

测试用例2: Prim算法测试

```
1  输入:
2  V=4, E=5
3  边信息:
4  (1,2,2), (1,3,2), (1,4,3), (2,3,4), (3,4,3)
5  预期输出: 7
```

测试用例3: Kruskal算法测试

```
1  输入:
2  V=4, E=5
3  边信息:
4  (1,2,2), (1,3,2), (1,4,3), (2,3,4), (3,4,3)
5  预期输出: 7
```

测试用例4: Huffman编码测试

1	输入:
2	n=4
3	概率: 0.100 0.100 0.200 0.600
4	预期输出: 1.600

4.3 测试结果

表1: 算法性能对比

算法名称	数据规模	时间复杂度	空间复杂度	正确率	适用场景
Dijkstra	$V \leq 5000, E \leq 2 \times 10^5$	$O(V^2)$	$O(V^2)$	100%	非负权图最短路径
Prim	$V \leq 5000, E \leq 2 \times 10^5$	$O(V^2)$	$O(V^2)$	100%	稠密图最小生成树
Kruskal	$V \leq 5000, E \leq 2 \times 10^5$	$O(E \log E)$	$O(E)$	100%	稀疏图最小生成树
Huffman	$n \leq 10^6$	$O(n \log n)$	$O(n)$	100%	数据压缩编码

表2: 测试用例详细结果

测试用例	算法	预期输出	实际输出	是否一致	备注
用例1	Dijkstra	2	2	是	基础功能正常
用例2	Prim	7	7	是	最小生成树正确
用例3	Kruskal	7	7	是	边排序合并正确
用例4	Huffman	1.600	1.600	是	编码长度计算正确

4.4 结果分析

- 正确性验证:** 所有算法在测试用例上均得到预期结果, 验证了算法的正确性
- 性能分析:** **Dijkstra算法:** 实现简单, 适合稠密图, 但在稀疏图上效率不如堆优化版本 **Prim算法:** 与Dijkstra类似, 适合稠密图的最小生成树问题 **Kruskal算法:** 基于排序和并查集, 适合稀疏图, 时间复杂度更优 **Huffman编码:** 使用线性选择而非堆结构, 在大数据量下效率有待优化
- 算法选择建议:** 最短路径问题: 小规模图使用朴素Dijkstra, 大规模稀疏图建议使用堆优化 最小生成树问题: 稠密图用Prim算法, 稀疏图用Kruskal算法 数据压缩: Huffman编码是经典最优前缀码算法, 实际应用可结合堆优化
- 改进方向:** Dijkstra和Prim算法可引入优先队列优化至 $O(E \log V)$ Huffman编码可使用最小堆实现更高效的节点选择 增加输入验证和错误处理机制